Ejercicios

Ejercicios adicionales: Productores y consumidores propios en Kinesis Data Streams

Complementa: Taller de Kinesis en AWS Academy (Learner Lab) **Región:** us-east-1 · **Rol:** `LabRole` · **Lenguaje:** Python + boto3 **Regla de oro:** al terminar cada ejercicio, elimine el stream (`aws kinesis delete-stream --stream-name <nombre> --enforce-consumer-deletion`). Más shards significa más costo por hora.

Antes de empezar: dos tipos de "competencia" (aclarar en clase)

Con la API directa (`GetRecords`), **los consumidores no se reparten los registros**: cada consumidor que lee un shard recibe *todos* los registros de ese shard. Lo que sí ocurre es que **compiten por la capacidad de lectura** del shard.

Concepto	Límite por shard (modo provisionado)
Lectura compartida entre todos los consumidores	2 MB/s
Llamadas `GetRecords` entre todos los consumidores	5 por segundo
Escritura (productores)	1 MB/s o 1000 registros/s

Por tanto:

- **Competencia por capacidad:** varios consumidores leyendo el mismo shard se estorban (errores `ProvisionedThroughputExceededException`).
- **Reparto del trabajo:** para que cada consumidor procese una parte distinta, se asigna **un shard por consumidor**. Con 1 shard solo un consumidor del grupo puede trabajar; añadir shards permite añadir consumidores sin que compitan. Esto es lo que hace automáticamente la librería KCL con sus "leases".

Ejercicio 1: Escriba su propio productor parametrizable (30 min)

Objetivo: controlar tasa de envío, cantidad de claves y sesgo, y manejar fallos parciales de `put_records`.

Especificación. Cree `productor_param.py` con estos argumentos:

Argumento	Significado
`--stream`	Nombre del stream
`--tasa`	Registros por segundo
`--duracion`	Segundos que corre
`--claves`	Número de partition keys distintas
`--sesgo`	Probabilidad (0 a 1) de usar siempre la clave `CT-001`

Requisitos

1. Enviar en lotes de máximo 500 registros con `put_records`.
2. `put_records` puede fallar parcialmente: revise `FailedRecordCount` y reenvíe **solo** los registros fallidos, con *backoff* exponencial.
3. Imprimir cada segundo: enviados, reintentados y fallidos.

Preguntas

- ¿Por qué no basta con revisar si la llamada lanzó excepción?
- ¿Qué pasa con el orden de los registros al reintentar?

Ejercicio 2: Consumidores que compiten por un mismo shard (30 min)

Objetivo: ver la competencia por capacidad de lectura en un shard.

Montaje

1. Cree el stream: `aws kinesis create-stream --stream-name ej-compite --shard-count 1`
2. Escriba `consumidor_shard.py` (especificación abajo).
3. Productor: `--tasa 200 --duracion 300 --claves 8`.

Especificación del consumidor

- Argumentos: `--stream`, `--indice-shard`, `--pausa` (segundos entre lecturas), `--nombre`, `--desde` (`LATEST` o `TRIM_HORIZON`).
- Debe contar registros recibidos por segundo y cuántas veces recibió `ProvisionedThroughputExceededException`, e imprimirlo cada segundo.
- Ante la excepción de límite, esperar un poco y reintentar (sin caerse).

Experimento

Corrida	Consumidores	`--pausa`	Hipótesis a verificar
A	1	1.0	Sin errores
B	3	1.0	3 llamadas/s en total: sin errores
C	3	0.4	≈ 7.5 llamadas/s > 5: aparecen errores

Entregable: tabla con registros/s recibidos y número de errores por consumidor en cada corrida.

Preguntas

- En la corrida B, ¿recibe cada consumidor todos los registros o solo una parte? ¿Por qué?
- Si el objetivo es *dividir el trabajo* entre 3 procesos, ¿sirve tener 3 consumidores sobre 1 shard?
- ¿Qué métrica de CloudWatch mostraría esta saturación? (`ReadProvisionedThroughputExceeded`.)

Ejercicio 3: Consumidores que no compiten, un shard por consumidor (30 min)

Objetivo: repartir el trabajo y demostrar que más shards permiten más consumidores útiles.

Montaje

1. Cree el stream: `aws kinesis create-stream --stream-name ej-reparto --shard-count 3`
2. Use el mismo `consumidor_shard.py`, pero lance **3 instancias**, cada una con `--indice-shard 0`, `1` y `2`, `--pausa 0.4`.
3. Productor: `--tasa 300 --claves 30` (muchas claves para repartir bien).

Entregable

- Registros por segundo de cada consumidor (deben ser similares y sumar la tasa del productor).
- Comparación contra el Ejercicio 2, corrida C: **¿siguen apareciendo errores de límite?**

Preguntas

- ¿Cuántos consumidores útiles puede tener un grupo con 3 shards? ¿Y si lanza un cuarto?
- ¿Por qué la elección de la clave de partición importa para que el reparto sea parejo?

Ejercicio 4: Shard caliente (hot shard) (25 min)

Objetivo: mostrar que tener más shards no ayuda si la clave está mal elegida.

Montaje: use el stream `ej-reparto` (3 shards) del Ejercicio 3.

1. Escriba `contador_por_shard.py`: lee **todos** los shards desde `TRIM_HORIZON` y cuenta cuántos registros hay por shard.
2. Corrida A: productor con `--sesgo 0 --claves 30`. Ejecute el contador.
3. Corrida B: productor con `--sesgo 0.8 --claves 30` (80 % de los eventos van a `CT-001`). Ejecute el contador.

Entregable: gráfico de barras con registros por shard en cada corrida.

Preguntas

- ¿Qué porcentaje del tráfico recibe el shard "caliente" en la corrida B?
- Proponga dos soluciones (por ejemplo, agregar un sufijo aleatorio a la clave o cambiar la granularidad de la clave) y explique qué orden se pierde en cada una.

Ejercicio 5: Reshard en caliente (40 min)

Objetivo: aumentar shards sobre un stream que ya tiene datos y entender los shards padre e hijo.

Montaje

1. Cree el stream: `aws kinesis create-stream --stream-name ej-reshard --shard-count 2`
2. Envíe datos durante un par de minutos.
3. Escale:

   ```
   kinesis.update_shard_count(StreamName="ej-reshard",
                              TargetShardCount=4,
                              ScalingType="UNIFORM_SCALING")
   ```

   Espere a que el estado vuelva a `ACTIVE`. Una sola llamada no puede más que duplicar el número de shards, y hay límites de operaciones de reshard por periodo; no lo repita sin necesidad.
4. Liste los shards con `list_shards` y observe cuáles tienen `EndingSequenceNumber` (**cerrados**, los padres) y cuáles no (**abiertos**, los hijos con `ParentShardId`).

Actividad

- Modifique su consumidor para que, al agotar un shard cerrado (cuando `NextShardIterator` llega vacío), **continúe con sus shards hijos**.
- Verifique que no pierde registros y que el orden por clave se mantiene: primero se lee el padre completo y luego el hijo.

Preguntas

- ¿Qué pasa con los datos que ya estaban en los shards padre?
- ¿Por qué se debe leer el padre hasta el final antes de leer los hijos?
- ¿Cuándo preferiría `split_shard` en lugar de `update_shard_count`?

Ejercicio 6 (opcional): Enhanced fan-out (30 min)

Objetivo: eliminar la competencia de lectura con capacidad dedicada por consumidor.

Pasos

1. En el stream `ej-compite` (1 shard), registre dos consumidores: `register_stream_consumer`.
2. Espere a que cada uno esté `ACTIVE` (`describe_stream_consumer`).
3. Lea con `subscribe_to_shard`: devuelve un flujo de eventos (HTTP/2) con registros; la suscripción dura unos 5 minutos y hay que renovarla.
4. Repita el experimento del Ejercicio 2, corrida C, con dos consumidores enhanced fan-out. ¿Siguen los errores?

Advertencias: tiene costo adicional (por consumidor-shard-hora y por GB leído) y su disponibilidad en el Learner Lab debe verificarse. Si falla por permisos, omita el ejercicio y deje el análisis teórico.

Reto final: Su propio "grupo de consumidores" con leases en DynamoDB

Una librería como KCL reparte shards automáticamente entre procesos. Implemente una versión mínima.

Diseño sugerido

- Tabla DynamoDB `leases` con clave `shard_id` y atributos: `propietario`, `expira_en`, `checkpoint` (último `SequenceNumber` procesado).
- Al arrancar, cada consumidor:
  1. Lista los shards abiertos.
  2. Intenta "adueñarse" de los que estén libres o vencidos con una **escritura condicional** (`ConditionExpression`), de modo que solo uno gane.
  3. Lee sus shards y guarda `checkpoint` cada N registros.
  4. Renueva su lease periódicamente; si se cae, otro lo toma y continúa desde el checkpoint.

Pruebas

- Con 4 shards y 2 consumidores, ¿cada uno toma 2?
- Mate un consumidor: ¿el otro toma sus shards?
- Reinicie un consumidor: ¿continúa desde el checkpoint sin repetir muchos registros? (Recuerde: entrega *al menos una vez*.)